QueryAPIGate HOW-TO GUIDE 49 · AGENTS

Let an AI agent write - with a person approving each write

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=49-let-an-agent-write-with-a-person-approving-each-write

Time: 20 minutes. **You'll end up with:** a support agent that opens tickets by itself, and closes one only when a person has approved that exact change - each write limited to the rows it may change, run once, and on record with who asked and who approved.

> **Stable** - write tools over MCP are under the compatibility promise since 0.30 (what that means).

Every command and response below was run against a real server, its MCP server and a PostgreSQL 16 `tickets` table, with a real MCP client (the Python SDK) as the agent - the Console at `http://localhost:5098` in this run.

How it works

Agents read only, unless a saved query's author marks it for agents **and** the agent's key may write through it. Then it's a write tool, in one of two modes:

- **`insert`** - one `INSERT`, run at once. It can add rows, never change or remove one.
- **`approval`** - the first call writes nothing and answers with a link; a person approves that exact write; the agent's second call runs it, once.

Either way the write runs in a transaction, rolled back if it changed more rows than the query allows.

Step 1: Mark the queries agents may write through

On the server, writes must be on (`QUERYAPIGATE_ALLOW_WRITES=1`), and `QUERYAPIGATE_CONSOLE_URL` makes the approval links full addresses:

```shell
curl -X POST http://localhost:5098/api/v1/queries -H "X-API-Key: $TOKEN" -H 'Content-Type:
application/json' -d '{
  "name": "file_ticket", "connection_name": "helpdesk", "description": "Open a ticket for a customer",
"publish": true,
  "sql": "INSERT INTO tickets (customer, title) VALUES (:customer, :title) RETURNING id, status",
  "parameters": {"customer": {"type": "string"}, "title": {"type": "string", "max_length": 200}},
  "agent_writes": "insert"
}'
curl -X POST http://localhost:5098/api/v1/queries -H "X-API-Key: $TOKEN" -H 'Content-Type:
application/json' -d '{
  "name": "close_ticket", "connection_name": "helpdesk", "description": "Close a ticket", "publish":
true,
  "sql": "UPDATE tickets SET status = '\''closed'\'' WHERE id = :id",
  "parameters": {"id": {"type": "integer", "min": 1}},
  "agent_writes": "approval"
}'
```

- **`max_affected_rows`** is how many rows one write may change - 1 unless you say more.
- Only one `INSERT`, `UPDATE` or `DELETE` can be marked - never DDL. `"sql": "TRUNCATE tickets"` with `agent_writes`: *An agent-writable query must be an INSERT, UPDATE or DELETE - one statement that changes data and nothing else.*
- PostgreSQL, MySQL, SQLite and DuckDB. On MySQL, keep these queries on InnoDB tables: a MyISAM table can't be rolled back, so the row limit couldn't undo a write to one.

Step 2: A key for the agent, allowed these writes

The same per-query write grant REST uses - and give the agent **its own key**, never an approver's token (Step 6 says why):

```shell
curl -X POST http://localhost:5098/api/v1/api-keys -H "X-API-Key: $TOKEN" -H 'Content-Type:
application/json' -d '{
  "name": "support-bot", "connections": [],
  "queries": [{"name": "file_ticket", "allow_writes": true}, {"name": "close_ticket", "allow_writes":
true}]
}'
```

The agent now sees them as tools, marked for what they do:

```text
close_ticket  readOnly False  destructive True   "This tool writes. A person approves each call before
it runs ... It changes at most 1 row."
file_ticket   readOnly False  destructive False  "This tool writes. It only adds rows, and runs without
an approval. It changes at most 1 row."
```

Step 3: It opens a ticket by itself

```text
file_ticket {"customer": "acme", "title": "Monitor flickers", "idempotency_key": "chat-81-1"}
→ {"status": "done", "request": "wr_ad9e595585bb", "rows_affected": 1, "rows": [{"id": 5, "status": "open"}]}
```

Called again with the same `idempotency_key` - an agent retrying after a timeout - it returns the same result, and there's still one ticket 5. The same key with other arguments is refused (`idempotency_conflict`).

Step 4: It asks to close one - and a person decides

```text
close_ticket {"id": 2}
→ {"status": "approval_required", "request": "wr_4cfa357ce998",
    "approve_at": "http://localhost:5098/console/approvals/wr_4cfa357ce998", "expires_at": "2026-10-11 16:12:47",
    "message": "A person must approve this write before it runs. Ask them to open http://localhost:5098/console/approvals/wr_4cfa357ce998 - then call this tool again with the same arguments and approval: \"wr_4cfa357ce998\". ..."}
```

Nothing has changed yet. The agent shows its user the link; it opens on the Console's **Approvals** screen (Observability → Approvals, with a count in the sidebar): who asked, the query, the SQL, **the exact arguments**, at most how many rows, when it expires - **Approve**, or **Reject** with a reason the agent is told. Owners and admins can decide (`writes.approve`); here, maria:

```shell
curl -X POST http://localhost:5098/api/v1/write-requests/wr_4cfa357ce998/approve -H "X-API-Key: $MARIA_TOKEN"
# {"id": "wr_4cfa357ce998", "status": "approved", "decided_by": "maria", "decided_via": "token", ...}
```

Step 5: It runs - that write, once

```text
close_ticket {"id": 2, "approval": "wr_4cfa357ce998"}
→ {"status": "done", "request": "wr_4cfa357ce998", "rows_affected": 1}
close_ticket {"id": 3, "approval": "wr_4cfa357ce998"}
→ {"error": "These arguments are not the ones approved - an approval is for exactly one write. ...", "code": "approval_mismatch"}
```

An approval is for exactly the arguments shown, by this caller, before it expires (15 minutes, `QUERYAPIGATE_WRITE_APPROVAL_MINUTES` - and once approved, the agent has that long again to run it). Calling again with the same approval returns the first result: a retry never writes twice.

When a write would change more than it may

`close_customer_tickets` - `UPDATE tickets SET status = 'closed' WHERE customer = :customer`, `max_affected_rows: 2` - asked for and approved for `acme`, who has four open tickets:

```text
→ {"error": "This write would have changed 4 rows, and this query may change at most 2 - nothing was changed",
    "code": "too_many_rows_affected"}
```

It ran in a transaction, saw 4 rows, and rolled back: all four tickets are as they were. The limit is checked when it runs, so a write approved when it would change one row, and that would change hundreds by then, is still stopped.

Step 6: Why the agent has its own key

If the agent were given maria's token, the write it asks for would be under her name - and approving it with that same token is refused:

```json
{"error": "This write was asked for under your own name - another approver must decide it", "code": "own_write"}
```

So a client configured with an approver's credential can't approve whatever its model asks for. Another approver must.

What's on record

Every request, 30 days - `GET /api/v1/write-requests/wr_4cfa357ce998`:

```json
{"id": "wr_4cfa357ce998", "query": "close_ticket", "version": 1, "caller": "support-bot", "status": "done",
 "connection": "helpdesk", "sql": "UPDATE tickets SET status = 'closed' WHERE id = :id", "arguments": {"id": 2},
 "max_affected_rows": 1, "decided_by": "maria", "decided_via": "token", "result": {"rows_affected": 1, "rows": []}}
```

Run history, under the agent, with who approved it - `GET /api/v1/history?key=support-bot`:

```json
{"query": "close_customer_tickets", "status": "error", "request": "wr_27d095cbc063", "approved_by": "maria", "transport": "mcp", "code": "too_many_rows_affected"}
{"query": "close_ticket", "status": "success", "rows": 1, "request": "wr_4cfa357ce998", "approved_by": "maria", "transport": "mcp"}
{"query": "file_ticket", "status": "success", "rows": 1, "request": "wr_ad9e595585bb", "approved_by": null, "transport": "mcp"}
```

And the audit log - the arguments by name and hash, never their values, since auditors read it and have no data access:

```json
{"actor": "maria", "via": "token", "action": "approve_write", "target": "wr_4cfa357ce998",
 "changes": {"query": "close_ticket", "caller": "support-bot", "arguments": ["id"], "arguments_hash": "5aa658ae..."}}
```

Good to know

- **What it can't judge** is intent: an approver who approves without reading approves what they see. The screen shows everything the write will do except which rows – `max_affected_rows` bounds how many.
- **Writes waiting** raise the `writes_awaiting_approval` alert; unapproved, they expire. An agent with 100 waiting is refused more (`too_many_pending_writes`).
- **Taking it back:** unmark the query (`agent_writes` removed in a new version) or narrow the key's grant, and the next call - even one with an approval - is refused. `execute_sql` never writes, whatever the key may do.
- **What a write returns** (`RETURNING`) is masked like any read, for a key with masking rules.
- The **Agents** screen counts each agent's write requests by how they ended.

Next steps

- Let AI clients sign in through your identity provider - so the agent writes as the signed-in person.
- Sign in to the Console through your identity provider - so approvers sign in with your MFA.
- MCP reference: Write tools.